DOCUMENTO DE LOGICA DE NEGOCIO

Central de facturacion con dias habiles

Integracion de POO + n8n | Costa Rica | Laboratorio #1

Estudiante: _____ Fecha: 19 de octubre de 2026

1. Problema y oportunidad

Una empresa necesita controlar facturas que tienen un plazo de pago definido en dias habiles. Si el vencimiento se calcula contando dias naturales, la fecha puede resultar incorrecta porque sabados, domingos y feriados no deben descontarse del plazo. Revisar cada factura manualmente consume tiempo y aumenta el riesgo de errores y de cobrar tarde.

La solucion consulta las facturas almacenadas en JSON Server y los feriados de Costa Rica en Nager.Date. n8n coordina las consultas, aplica las reglas de negocio en JavaScript y actualiza un registro solamente cuando el estado o la fecha de vencimiento deben cambiar.

2. Recorrido de los datos

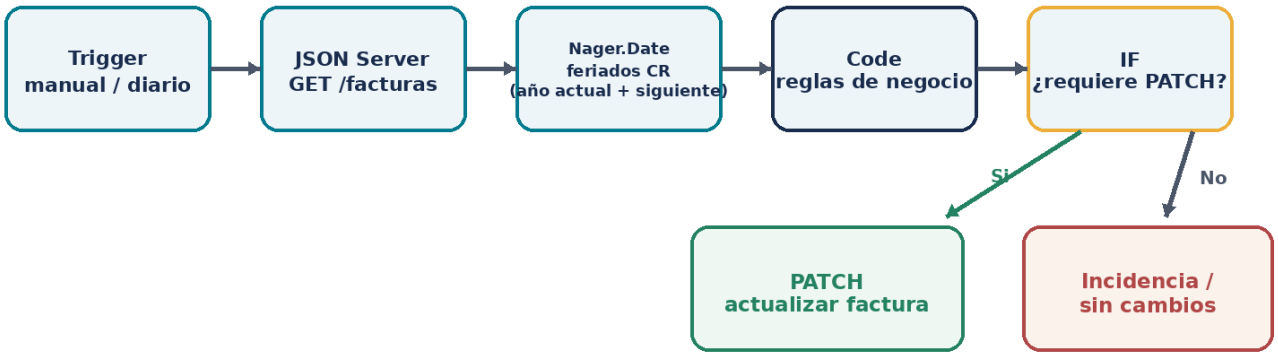


Figura 1. Flujo de ejecucion y punto de control que evita actualizar facturas ante errores.

El workflow dispone de un disparador manual para pruebas y uno diario a las 08:00, con zona horaria America/Costa_Rica. El nodo HTTP Request consulta JSON Server; dos consultas a Nager.Date recuperan los feriados del año actual y del siguiente. El nodo Code valida y calcula. El IF envia a PATCH solo los resultados que requieren cambios; los errores y registros sin cambios terminan en una ruta separada.

3. Reglas de negocio implementadas

Regla	Implementacion	Evidencia en la ejecucion
1. Plazo en dias habiles	Se inicia el conteo al dia siguiente de la emision y se cuentan solo dias habiles.	Fecha de vencimiento calculada.
2. Fines de semana	Sabado y domingo nunca suman al plazo.	El calendario ignora dias con getUTCDay 0 o 6.
3. Feriados nacionales	Las fechas recibidas de Nager.Date se excluyen del conteo.	Consultas externas para CR en el año actual y siguiente.
4. Solo pendientes	Pagadas u otros estados no se recalculan.	Resultado IGNORAR; no pasa al PATCH.
5. Validacion	ID, fecha ISO, dias habiles entre 1 y 365, monto positivo y estado obligatorio.	Los errores producen ERROR_DATOS.
6. Vencimiento	Una factura vence cuando hoy es posterior a su fecha limite. Si vence hoy, sigue pendiente.	Estado calculado pendiente o vencida.
7. Evitar actualizaciones inutiles	Si vencimiento y estado ya coinciden con el calculo, no hay PATCH.	Resultado SIN_CAMBIOS.
8. Fallos seguros	Si JSON Server o la API de feriados falla, no se modifica ninguna factura.	ERROR_JSON_SERVER o ERROR_API_FERIADOS.

4. Modelo orientado a objetos

Clase	Responsabilidad	Metodos principales
CalendarioLaboral	Mantiene un conjunto de feriados, identifica dias habiles y suma dias al plazo.	esDiaHabil(); sumarDiasHabiles(); esFechaISO().
Factura	Representa una factura, valida sus campos, calcula el vencimiento y determina si ya vencio.	validar(); calcularFechaVencimiento(); evaluar().
CuentaPorCobrar	Evalua el lote de facturas y captura errores por registro sin detener el resto.	evaluarPendientes().

La demostracion POO esta separada de n8n: `node main.js` instancia las clases con las facturas de `db.json`, muestra decisiones por consola y no requiere un workflow activo. `npm test` prueba el calculo de fechas, feriados, validacion, estados y errores.

5. Correspondencia entre las clases y el nodo Code

El Code node contiene una version autocontenida del mismo algoritmo de calendario y evaluacion de factura. No importa archivos del equipo, porque el workflow exportado debe poder importarse y ejecutarse sin depender de rutas locales. La regla para sumar dias, validar datos, definir el estado y decidir si hace falta PATCH se mantiene equivalente a `CalendarioLaboral`, `Factura` y `CuentaPorCobrar`. Los archivos de clases sirven como version legible y testeable; el Code aplica esa misma politica sobre la respuesta HTTP real.

6. Manejo de errores

Situacion	Respuesta del sistema
Dato incompleto	La factura F-2026-004 tiene diasHabiles igual a 0. Se devuelve ERROR_DATOS y el IF la envia a la ruta de incidencias, por lo que no se ejecuta PATCH.
API externa caida	Los HTTP Request tienen reintentos y continuacion controlada ante error. Si una respuesta de feriados no contiene fechas validas, el Code genera ERROR_API_FERIADOS y bloquea todas las actualizaciones.
JSON Server fuera de servicio	Si no se puede leer la cartera, el Code devuelve ERROR_JSON_SERVER. No intenta calcular ni guardar cambios.
Factura pagada o sin cambios	La factura pagada produce IGNORAR; una factura cuyo vencimiento ya coincide produce SIN_CAMBIOS. Ambos resultados evitan escrituras innecesarias.

7. Pruebas y demostracion

- Ejecutar `npm run demo` y explicar una factura vencida, una pendiente y una entrada invalida.
- Ejecutar `npm test`; el resultado esperado es `OK: 6 pruebas pasaron.`.
- Iniciar JSON Server con `npm run api` y verificar la API con `curl http://localhost:3000/facturas`.
- Importar el JSON del workflow en n8n, ejecutarlo manualmente y revisar la salida del nodo Code, la bifurcacion IF y el PATCH.
- Probar un error externo cambiando temporalmente una URL de feriados por una ruta invalida; verificar ERROR_API_FERIADOS y restaurar la URL.

8. Decisiones tecnicas y conclusion

Las fechas se manejan como valores ISO `AAAA-MM-DD` y las operaciones de calendario se realizan en UTC al mediodia para evitar efectos de horario. La fecha de hoy se obtiene con la zona America/Costa_Rica. Se consultan dos años de feriados para cubrir vencimientos que crucen de año. Se usa PATCH para modificar solo los campos de una factura existente y el flujo evita cambios cuando no son necesarios.

El principal reto es que POO y automatizacion compartan las mismas reglas a pesar de ejecutarse en entornos distintos. La separacion por clases permite probar la logica antes de integrarla; el workflow coordina las fuentes externas y aplica controles para no escribir informacion incompleta o calculada sin feriados confiables.

Referencias tecnicas

- Nager.Date Public Holiday API: https://date.nager.at/nl/Api
- JSON Server: https://github.com/typicode/json-server
- n8n HTTP Request: https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.httprequest/
- n8n Code node: https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.code/